

<u>CL-2006</u> <u>Operating System Lab</u>	<u>LAB -05</u> <u>InterProcess Communication</u> <u>Using Pipes and Shared</u> <u>Memory</u>
---	---

National University of Computer and Emerging Science
Spring 2026

Table of Content

Objective.....	3
InterProcess Communication.....	3
Pipe.....	3
Named Pipe.....	4
Shared Memory.....	4
For Server.....	5
For Client.....	6
Lab Activity.....	6

Objective

After completing this lab, you will be able to:

- Demonstrate System Calls Programmatically
- Learn and Understand InterProcess Communication using implementation of Pipes.
- Develop an understanding of system calls by implementing various functions programmatically and explore the functionality and usage of each system call through practical coding exercises, emphasizing their role in ordinary pipes, named pipes and shared memory.
- Implement C program producing and consuming data using ordinary and named pipes and shared memory by utilizing appropriate system calls.

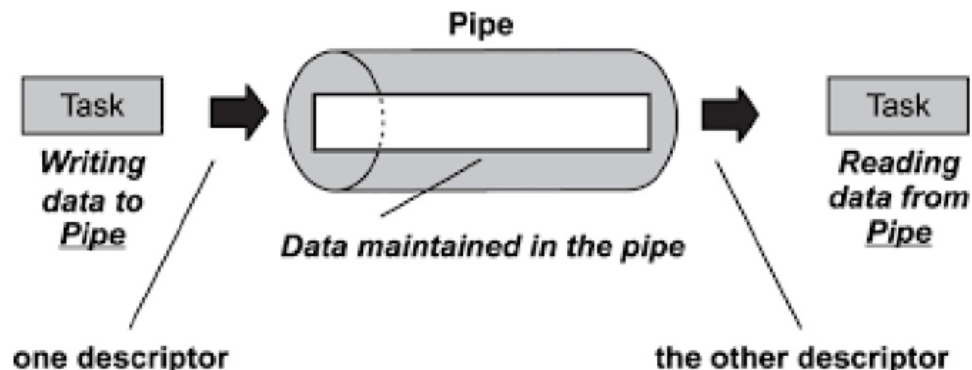
InterProcess Communication

IPC is very vital in any Embedded System. A program may have to feed another process for it to proceed. It is inherent in all the embedded systems. Following are very commonly used IPC mechanisms.

- Pipe
- Named Pipe
- Signals
- Shared Memory
- Semaphores
- Remote procedure calls
- Sockets

Pipe

Ordinary pipes allow two processes to communicate in standard producer consumer fashion. The producer writes to one end of the pipe (the **write-end**) and the consumer reads from the other end (the **read-end**). As a result, ordinary pipes are unidirectional, allowing only one-way communication. If two-way communication is required, two pipes must be used, with each pipe sending data in a different direction. Data written to the write end of a pipe can be read from the rear end of the pipe. Another thing to remember is that pipes can only be used between related processes. No two different unrelated processes can use pipe. This is the case here.

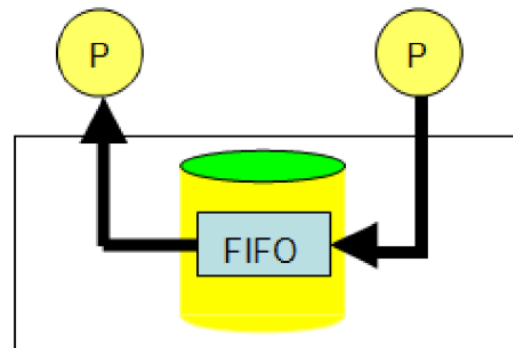


PIPE can be created with **pipe() system call** and it will return two file descriptors accepting an array of integers as an argument. One **file descriptor (FD)** will be used as a read end file descriptor and the second one can be used as a Write end file descriptor. File descriptor is an integer allotted by the system for each file that is created.

Walk through the code example in unnamedpipe.c and see that **Hello Mr.Linux** is sent to the child process which the user can see on the screen. One beauty in this mechanism is unless the parent writes the child cannot read and there exists a synchronization which is very vital. The only disadvantage associated with pipe is, **it can be used for only related processes** (A process when has a child for itself then they become related). Now this problem can be overcome by using Named pipe or FIFO.

Named Pipe

To overcome that we can use 'Named Pipes' which is also known as **FIFO (First In First Out)**. Here the concept is slightly different. Taking a real world example again: suppose a person has to pass a letter to someone. Due to some situations, it cannot be given in person. A simple solution is to find a third person who is familiar to both the people. Now that third person will be able to hand over the paper to the destination successfully. Same is the case with named pipes. It can be used for communication between two different processes.



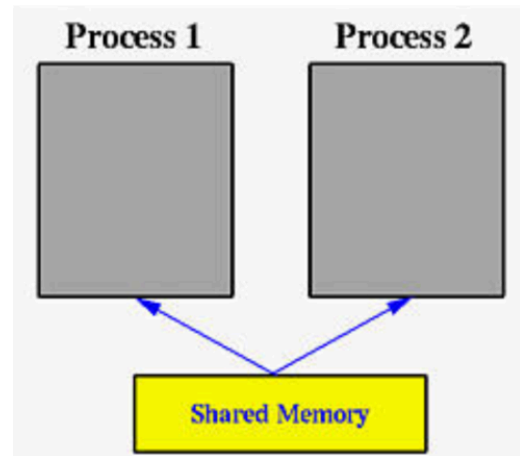
The sequence goes like this, Process A will write the data in a common file which Process B can also access. After data has been written by A, B will read the data from that common file. After reading the file can be deleted. The term file has to be refined. It is also called FIFO in Linux which can be created with available system calls. The system call **mkfifo** can be used to create a FIFO. In FIFO, two different processes can communicate which is revealed with the following given C code, where `fifo_write.c` is FIFO write program and `fifo_read.c` is read program.

The write program has to be executed first then read can be executed. Even if the user executes a read program, it will wait for the writer to write the data. So, here exists an auto synchronization which is a highly appreciable feature. Reading from or writing to a named pipe occurs just like traditional file reading and writing; except that the data for named pipe is never written to or read from a file in hard disk but memory.

Shared Memory

In the discussion of the `fork ( )` system call, we mentioned that a parent and its children have separate address spaces. While this would provide a more secure way of executing parent and children processes (because they will not interfere with each other), they shared nothing and

have no way to communicate with each other. A shared memory is an extra piece of memory that is attached to some address spaces for their owners to use. As a result, all of these processes share the same memory segment and have access to it. Consequently, race conditions may occur if memory accesses are not handled properly. The following figure shows two processes and their address spaces. The yellow rectangle is a shared memory attached to both address spaces and both process 1 and process 2 can have access to this shared memory as if the shared memory is part of its own address space. In some sense, the original address space is "extended" by attaching this shared memory.



This mechanism is very important and most frequently used. Shared memory can even be used between unrelated processes. By default page memory of 4KB would be allocated as shared memory. Assume process 1 wants to access its shared memory area. It has to get attached to it first. Though its P1's memory area, it cannot get access as such. Only after attaching it can gain access. A process creates a shared memory segment using `shmget()`. The original owner of a shared memory segment can assign ownership to another user with `shmctl()`. It can also revoke this assignment. Other processes with proper permission can perform various control functions on the shared memory segment using `shmctl()`.

Once created a shared memory segment can be attached to a process address space using `shmat()`. It can be detached using `shmdt()`. The attaching process must be appropriate permissions for `shmat()`. Once attached, the process can read and write segments, as allowed by the permission requested in the attach operation. A shared memory segment can be attached multiple times by the same process. A shared memory segment is described by a control structure with a unique ID that points to an area of physical memory. The identifier of the segment is called the `shmid`. The structure definition for the shared memory segment control structure and prototypes can be found in `<sys/shm.h>`.

There are three steps:

1. Initialization
2. Attach
3. Detach

The client server scenario would be perfect to demonstrate shared memory, the general scheme of using shared memory is the following:

For Server

1. Ask for a shared memory with a memory key and memorize the returned shared memory ID. This is performed by system call `shmget()`.
2. Attach this shared memory to the server's address space with system call `shmat()`.

3. Initialize the shared memory, if necessary.
4. Do something and wait for all clients' completion.
5. Detach the shared memory with system call `shmdt()`.
6. Remove the shared memory with system call `shmctl()`.

For Client

1. Ask for a shared memory with the same memory key and memorize the returned shared memory ID.
2. Attach this shared memory to the client's address space
3. Use the memory
4. Detach all shared memory segments, if necessary
5. Exit.

Lab Activity

1. Reverse the example in 'unnamed_pipe.c' so that child would send message to parent and the parent would print the message on screen.
2. Run the FIFO example with three read processes and one write process.
3. A software development team is working on an inter-process communication (IPC) system where a parent process must communicate with a child process using shared memory. They need a solution where:
 - The parent process creates a shared memory segment.
 - The child process writes a message to the shared memory.
 - The parent process reads the message after a short delay to ensure synchronization.
 - After reading the data, both processes clean up the shared memory properly.
4. Create a program where the parent process accepts a number from the user and sends to the child process. In the child process, each digit of the number is separated and sent back to the parent process through separate pipes. After receiving all digits, the parent process calculates and displays the product of all the digits.